PORTREGISTRY

Product Requirements Document

A local-first, cross-platform port discovery and service registry

Document	Value
Version	1.0 — First-pass implementation baseline
Date	9 November 2026
Status	Implementation-oriented product specification
Primary delivery	Working local demo: daemon + discovery + registry + GUI/TUI/CLI
Initial deployment	Single machine, local SQLite database, offline-capable

This document is the source of truth for the first implementation. If a requirement is ambiguous, do not silently invent product behaviour: record the ambiguity and ask the product owner before making a consequential choice.

1. Executive summary

PortRegistry is a local-first application for discovering listening TCP and UDP endpoints, identifying the processes that own them, and maintaining a human-managed registry of services and logical port reservations. It correlates observed machine state with declared intent so users can identify occupied reservations, unregistered listeners, service mismatches, and incomplete discovery.

The product is intended to be useful on a developer workstation or homelab without requiring a cloud account, external database, or hosted control plane. It provides three interfaces—Django web GUI, Textual TUI, and Typer CLI—all backed by one local daemon. The daemon is the single owner of discovery, scheduling, application logic, and SQLite writes.

The first release is a demonstrable single-machine product. It includes native socket discovery, optional Docker discovery, a persistent service registry, reservations, conflict analysis, meaningful event history, and usable interfaces. Multi-machine remote agents and process/container control are explicitly outside the first release.

2. Product goals and non-goals

2.1 Goals

- Discover TCP listening sockets and UDP bound sockets, including IPv4 and IPv6 where the operating system and permissions permit.
- Resolve socket ownership to process instances and collect useful process metadata without requiring automatic elevation.
- Maintain user-defined services and advisory port reservations independently of transient process/socket observations.
- Detect and explain conflicts between observed endpoints, declared services, and reservations.
- Discover Docker containers and host-published port mappings when Docker is available.
- Offer equivalent core workflows through a browser GUI, terminal UI, and CLI.
- Run offline and locally with SQLite; keep all state on the user's machine by default.
- Retain current state and meaningful state changes without storing every polling snapshot.

2.2 Non-goals for MVP

- Killing processes, starting/stopping operating-system services, modifying firewall rules, or managing Docker containers.
- Remote agents, central server, multi-user collaboration, cloud synchronization, telemetry, or hosted accounts.
- Guaranteed reservation of operating-system ports. Reservations are advisory records only.
- Kernel-level or OS-specific real-time socket notifications across every supported platform.
- Full network monitoring, packet capture, traffic inspection, intrusion detection, or vulnerability scanning.
- Arbitrary commands inside containers or execution of discovered process command lines.

3. Target users and primary use cases

User / context	Primary need
Developer workstation user	Find what owns a port, understand why a development server cannot bind, and avoid port collisions.
Homelab operator	Inventory services running on a host, including Docker-published ports, and document intended ownership.
Systems / platform engineer	Inspect local process/socket state, identify unregistered listeners, and review changes over time.

3.1 Core user journeys

1. Start the daemon. It creates or opens the local database, initializes machine identity, performs an initial discovery, and exposes a loopback API.
2. Open the GUI or TUI. The user sees current endpoints, discovery freshness, Docker availability, services, reservations, and conflicts.
3. Select an endpoint. The user can inspect protocol, address, port, process identity, executable metadata, and associated Docker context when known.
4. Register a service. The user gives it a name, description, expected executable and/or expected endpoint, and optional notes.
5. Reserve a port. The system checks current observations and existing reservations, then records an advisory reservation and reports any conflict.
6. Review conflicts and history. The system explains whether a reservation is occupied, a listener is unregistered, or observed ownership differs from the declared service.

4. Product scope and release definition

4.1 MVP — required

- Single local machine and persistent machine UUID.
- Daemon process with single-instance protection, graceful shutdown, health/status endpoints, and periodic scheduler.
- Native TCP/UDP socket discovery with process resolution and normalized IPv4/IPv6 endpoint records.
- Hybrid reconciliation: periodic native scans as the correctness mechanism; Docker event stream plus periodic Docker inventory reconciliation.
- Optional Docker integration that degrades gracefully when Docker is absent or inaccessible.
- Persistent service CRUD and advisory port reservations.
- Conflict engine covering occupied reservations, overlapping reservations, unregistered listeners, service mismatch, missing service, and incomplete discovery.
- Meaningful history of state changes and discovery errors.
- Django web GUI, Textual TUI, and Typer CLI as daemon clients.
- Local HTTP/JSON API and Server-Sent Events (SSE) for live updates.
- Automated tests, setup instructions, and pip-installable package.

4.2 Explicitly deferred

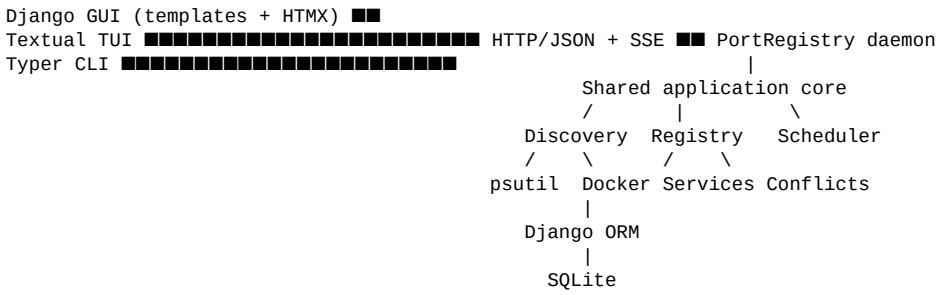
- Remote machines and agents; the data model may be machine-aware, but remote transport and enrollment are not MVP requirements.
- Native OS event subscriptions for process/socket changes.
- Advanced service dependency visualization and dependency health orchestration.
- OS service installation, launch-at-login, systemd/Windows Service/launchd integration. Initial daemon lifecycle is controlled by CLI.
- Process termination, Docker management, port binding, and privileged remediation.

4.3 Demo acceptance scenario

A successful demo must show the daemon starting, discovering a test TCP listener and a UDP-bound socket, resolving their owning processes where permissions allow, displaying them in the GUI and TUI, creating a service and reservation, detecting an occupied or overlapping reservation, discovering a Docker container and published mapping when Docker is available, and showing a meaningful history entry after a state change. The CLI must query the same daemon-backed state. Restarting the daemon must preserve services, reservations, machine identity, and history.

5. Architecture and technical decisions

Architectural invariant: the daemon owns the database, discovery, scheduling, event processing, and application logic. GUI, TUI, and CLI are clients. No client may open SQLite directly, run its own discovery loop, or independently mutate application state.



Area	Selected direction
Language/runtime	Python 3.12+ (verify compatibility at implementation time)
Web framework	Django 5.2 LTS, Django REST Framework, Django ASGI
ASGI server	Uvicorn; one daemon process / one worker for the initial architecture
Database	SQLite with WAL mode, foreign keys enabled, and a busy timeout
GUI	Django templates + HTMX + Bootstrap 5 or lightweight vanilla CSS/JS; no SPA requirement
TUI	Textual + Rich
CLI	Typer
Native discovery	psutil, with platform-specific diagnostic/fallback providers where useful
Docker	Docker SDK for Python; optional dependency extra
HTTP client	httpx for CLI/TUI daemon communication
Paths/config	platformdirs; validated local configuration
Testing/quality	pytest, pytest-django, Ruff; type checking where practical

These are implementation baselines, not permission to add heavyweight infrastructure. Do not introduce Redis, Celery, PostgreSQL, Kubernetes, a cloud service, or a separate frontend framework for the MVP.

6. Runtime, daemon, and client behaviour

6.1 Daemon responsibilities

- Acquire a single-instance lock and verify any stale PID-file information before assuming another daemon is running.
- Initialize configuration, data directories, database, migrations, machine identity, discovery providers, API server, and scheduler.
- Run an initial discovery and then scheduled reconciliations.
- Own all database writes and publish committed state-change events.
- Expose health, status, query, mutation, discovery, and event-stream API endpoints.
- Handle SIGINT/SIGTERM, stop accepting new work, close Docker event streams, stop scheduling, finish or safely cancel scans, flush writes, and release the lock.

6.2 Commands

```
portregistry daemon start
portregistry daemon stop
portregistry daemon status
portregistry daemon restart
portregistry daemon logs

portregistry gui
portregistry tui
portregistry scan
portregistry ports
portregistry conflicts
```

The daemon must support foreground execution for development and debugging. Normal CLI startup may detach it in the background. Initial release does not need native OS service installation.

6.3 API and networking

- Default bind address: 127.0.0.1:8765. Binding to non-loopback interfaces must require explicit configuration and a clear security warning.
- API prefix: /api/v1/. Provide a health endpoint that can be checked without exposing sensitive data.
- Use Django session authentication and CSRF protection for browser mutations. Use a randomly generated local API token for CLI/TUI clients, stored in a permission-restricted config file; never log it.
- Use SSE for event updates. Clients must reconnect and refresh authoritative state after a disconnect or missed events.
- Keep request validation, pagination, consistent error responses, and sensible request-size limits.
- Do not run multiple web workers that each initialize schedulers. The single daemon owns one scheduler.

7. Discovery requirements

7.1 Native socket discovery

Use psutil.net_connections(kind="inet") as the primary cross-platform discovery mechanism. Support TCP listeners and UDP sockets bound to local ports. TCP and UDP semantics must remain distinct: UDP has no TCP LISTEN state.

- Capture protocol, local address, local port, address family, socket state, PID when available, and observation timestamp.

- Resolve process metadata: PID, process name, executable path, command line (optional), username (optional), parent PID, and process creation time.
- Support IPv4, IPv6, wildcard addresses, loopback addresses, and multiple sockets/processes sharing a port where the OS permits.
- Treat permission-denied, process-exited-during-scan, and unavailable metadata as partial-resolution conditions, not fatal scan failures.
- Do not automatically invoke sudo, UAC elevation, or privilege escalation. Clearly disclose incomplete visibility.
- Never collect process environment variables. Never execute command lines or treat discovered metadata as trusted instructions.

7.2 Hybrid discovery model

A full periodic reconciliation is the source of truth for native socket state. psutil does not provide a portable, comprehensive socket-open/socket-close event stream. Do not claim real-time event-driven native discovery in the MVP.

Provider	MVP behaviour	Initial interval
Native sockets/processes	Periodic full scan; normalize and reconcile current state.	10 seconds, configurable
Docker inventory	Periodic container/port inventory reconciliation.	30 seconds, configurable
Docker events	Subscribe to container lifecycle events; trigger targeted refresh and reconcile.	Continuous stream with reconnect

- Never allow overlapping full scans for the same machine.
- An event may trigger targeted refresh, but periodic reconciliation remains the recovery mechanism.
- A failed or partial scan must not erase current state or imply that endpoints disappeared.
- Only mark an endpoint/process absent after a successful authoritative reconciliation for the relevant scope.
- Record scan start/end, status, counts, duration, provider errors, and resolution failures.

7.3 Platform limitations

- Linux: psutil primary; /proc or `ss -lntup` may be used as a diagnostic/fallback path.
- Windows: psutil primary; Windows IP Helper APIs may be considered as a later native fallback. PowerShell Get-NetTCPConnection is diagnostic, not the primary architecture.
- macOS: psutil primary; lsof may be diagnostic/fallback.
- Do not claim full platform parity until tested on actual Linux, Windows, and macOS systems.
- Permissions, OS socket APIs, IPv6 dual-stack semantics, and container virtualization can limit what can be observed.

8. Docker discovery

Docker integration is part of MVP but optional at runtime. Install it through an optional package extra, for example `pip install "portregistry[docker]"`. If the SDK, daemon, socket, or permissions are unavailable, native discovery must continue normally and the UI must show Docker as unavailable or degraded.

- Read container ID, name, image reference/ID, status, creation/start timestamps, labels, and network metadata.
- Read exposed internal container ports and published host mappings, including host IP, host port, container port, and TCP/UDP protocol.

- Subscribe to Docker lifecycle events and reconcile with periodic full inventory.
- Represent internal container ports, host-published mappings, and host-visible sockets as distinct concepts.
- Do not assume a host PID corresponds to the process inside a container. Docker Desktop on Windows/macOS uses virtualization and may not expose container processes as host processes.
- Use read-only Docker API operations. Never execute commands in containers, manage their lifecycle, or expose the Docker socket through PortRegistry's HTTP API.
- Explain that Docker socket access is highly privileged, even when PortRegistry only intends read operations.

9. Domain model and persistence

Separate persistent user intent from transient observed machine state. Services and reservations are user-defined records. Processes, sockets, scans, and Docker observations reflect what discovery currently sees.

Entity	Purpose / important fields
Machine	UUID, hostname, OS, OS version, architecture, is_local, last_seen, created_at.
ProcessInstance	Machine FK, PID, process creation time, process name, executable path, optional command line/user/parent PID, identity hash, first_seen, last_seen, active.
Socket	ProcessInstance FK nullable when unresolved, protocol, local address/port, address family, state, active, last_seen.
DiscoveryScan	Machine FK, start/end, status, socket count, resolved process count, failures, provider errors, duration.
SocketObservation	Scan FK, endpoint/process identity, timestamp; store only meaningful observations or deduplicate unchanged polling.
Service	Machine FK, name, description, category, expected executable, expected working directory, documentation URL, notes, timestamps.
PortReservation	Machine FK, optional Service FK, port, protocol, bind address, status, description, timestamps.
DockerContainer	Machine FK, container ID, name, image metadata, status, start/last-seen times, labels JSON.
DockerPortMapping	Container FK, internal port, host port, protocol, host IP, status.
DockerImage	Image ID, repository, tag, optional digest.
DockerNetwork	Container/network association and network metadata.
DockerEvent / ProcessEvent	Meaningful lifecycle and state-change records.
ServiceDependency	Future/P1 relationship between services; not required for MVP UI.

9.1 Identity and constraints

- Generate a PortRegistry machine UUID once and persist it. Do not use hardware serial numbers or sensitive hardware identifiers.
- Process identity is Machine UUID + PID + process creation timestamp. PID alone is not stable because it is reused.
- If process creation time is unavailable, mark identity as incomplete; do not fabricate a stable identity.
- Suggested unique constraint: (machine, pid, create_time) for ProcessInstance.
- Suggested unique constraint: (machine, container_id) for DockerContainer.
- Validate port range 1–65535 for normal reservations; port 0 is not a normal reservation.

- Use foreign keys and indexes for machine/port/protocol, process PID/name, service name, and scan chronology.

9.2 SQLite operating rules

- Enable WAL mode and foreign-key enforcement; configure a busy timeout.
- Only the daemon writes to the database. Keep write transactions short.
- Never hold a transaction open while querying operating-system APIs or Docker.
- Collect and normalize discovery results first, then compare and persist in a short transaction.
- SQLite WAL improves reader/writer coexistence; it does not turn SQLite into a multi-writer database.
- Use migrations and preserve user-defined services/reservations across upgrades and daemon restarts.

10. Services, reservations, and conflict engine

10.1 Service registry

Users can create, view, edit, and delete service definitions. A service may describe an application, daemon, database, homelab service, or development tool. Service metadata is independent of whether the process is currently running.

- Fields include name, description, category, machine, expected executable, expected working directory, documentation URL, notes, and optional associated reservations.
- A service may have zero, one, or multiple expected endpoints; do not hard-code a one-port-only assumption into the schema.
- Deleting a service must not silently delete historical events or unrelated observed sockets. Define and test reservation deletion/retention behaviour.

10.2 Port reservations

A reservation is a logical/advisory claim that a service intends to use a protocol/port/address combination. It does not bind the port, prevent another application from binding it, or guarantee future availability.

- Create, edit, release, list, and inspect reservations.
- Validate port, protocol, address, machine, and optional service association.
- Check current occupancy and existing reservations when creating/updating; report findings without claiming race-free OS enforcement.
- Support TCP and UDP separately. TCP/UDP on the same numeric port are not inherently conflicting.
- Reservation status should distinguish active, released, and conflict/attention states without losing history.

10.3 Conflict rules

- Definite conflict: same machine, same protocol, overlapping bind endpoint, and same port under known address-family semantics.
- Wildcard IPv4 address (0.0.0.0) overlaps specific IPv4 local addresses such as 127.0.0.1.
- IPv6 wildcard overlap with IPv4 depends on dual-stack socket options and OS behaviour; report potential conflict when this cannot be determined.
- TCP and UDP reservations on the same port are separate and do not conflict solely because the number matches.
- Detect occupied reservation, overlapping/duplicate reservation, unregistered listener, service mismatch, missing expected service, and incomplete discovery.
- Distinguish definite from potential conflicts. Include an explanation and evidence for every conflict.
- Do not report absence as certainty when discovery was partial or failed.

11. History and event model

Retain current state and meaningful state changes, not a new durable snapshot of every polling cycle. Deduplicate unchanged observations.

- Record endpoint/process/container appearance and disappearance, ownership changes, reservation creation/update/release, conflict appearance/resolution, machine online/offline, and discovery failures.
- Store event timestamp, machine, event type, related entity identifiers, and concise structured details.
- Proposed default retention: 30 days, configurable. Treat this as a default to confirm during implementation; do not hard-delete user-defined services/reservations as part of event cleanup.
- Emit SSE notifications only after the corresponding database transaction commits.

12. User interfaces

12.1 Django web GUI

Use Django server-rendered templates with HTMX for partial updates and Bootstrap 5 or lightweight custom styling. Do not build a React SPA.

Page	Required content / actions
Dashboard	Endpoint totals, TCP/UDP split, services, reservations, conflicts, Docker status, scan freshness, active ports, recent events.
Ports explorer	Search/filter by port, process, protocol, machine, state, address family; sorting and endpoint detail.
Process detail	Process identity, PID/start time, executable metadata, owned endpoints, first/last seen.
Services	List, create, edit, delete, service details, expected executable/endpoints, notes.
Reservations	List, create, edit, release, filter, current occupancy/conflict explanation.
Conflicts	Type, severity/certainty, machine, endpoint, expected service, observed owner, evidence and explanation.
Machines	Local machine identity, OS details, daemon status, last successful discovery.
History	Meaningful event timeline, filters, scan failures, state changes.

- Responsive, accessible, keyboard-friendly, information-dense layout with clear empty/loading/error/degraded states.
- Show scan freshness and permission limitations; do not present stale data as live.
- Use SSE/HTMX for live updates without requiring page reloads.
- Escape user-controlled values and use CSRF protection for mutations.

12.2 Textual TUI

- First-class interface for SSH/headless use; it must remain usable without a graphical desktop.
- Screens: Dashboard, Ports, Services, Reservations, Conflicts, Machines, History, Settings.
- Keyboard actions: q quit, r refresh, / search, a add service, n new reservation, Enter details, Esc back, Tab navigation, ? help.
- Use asynchronous API requests so refreshes do not freeze the UI.
- Use the daemon API only; no direct DB access or local discovery.

12.3 CLI

```
portregistry scan [--protocol tcp|udp] [--port PORT]
portregistry ports [--port PORT] [--protocol tcp|udp] [--machine MACHINE]
portregistry service list|add|show ID|edit ID|delete ID
portregistry reserve
portregistry reservations
portregistry release ID
portregistry conflicts
portregistry history
portregistry machines
portregistry report
```

Support readable tables plus JSON and CSV output where meaningful. Mutating commands must validate input and display the resulting state/conflicts. CLI is a daemon client, not a second implementation of discovery.

13. API contract

Method / route	Purpose
GET /api/v1/health/	Liveness/readiness summary; no sensitive details.
GET /api/v1/status/	Daemon, machine, scheduler, last scan, provider status.
GET /api/v1/machines/	List machine records.
GET /api/v1/ports/	Query normalized current endpoints with filters/pagination.
GET /api/v1/processes/	List process instances and metadata.
GET /api/v1/containers/	List Docker containers and mappings.
GET, POST /api/v1/services/	List/create service definitions.
GET, POST /api/v1/reservations/	List/create reservations.
GET /api/v1/conflicts/	List current conflict findings.
POST /api/v1/discovery/scan/	Request a scan; return accepted/current scan state.
GET /api/v1/history/	Query meaningful events.
GET /api/v1/events/stream/	SSE stream for committed state changes.

Provide detail/update/delete endpoints for mutable resources. Define consistent JSON error shapes, validation messages, pagination, and status codes. Discovery scan requests should be idempotent or coalesced when a scan is already running.

14. Security, privacy, and safety

- Loopback-only by default. Remote binding must be explicit and documented as a security-sensitive configuration.
- Use session auth + CSRF for browser requests and a random local token for CLI/TUI. Restrict config/database file permissions where supported.
- Never log API tokens or secrets. Do not collect process environment variables.
- Treat process names, command lines, paths, Docker labels, service names, and notes as untrusted input. Escape them in HTML and validate API payloads.
- Never execute discovered command lines, invoke a shell using unsanitized values, or run arbitrary container commands.
- Do not auto-elevate privileges or bypass operating-system permissions.

- Do not expose Docker socket access through the PortRegistry API. Clearly explain Docker socket privilege implications.
- No telemetry, cloud account, or outbound network requirement for core functionality.

15. Reliability and failure handling

- Daemon startup must be safe when the DB already exists, after an unclean shutdown, and when Docker is unavailable.
- A provider failure must not prevent other providers from operating.
- A partial scan must preserve the last known state for scopes that were not successfully observed and clearly mark freshness/completeness.
- If a process exits while metadata is being read, record a partial resolution and continue.
- Docker event stream disconnects must trigger reconnect with backoff and a full inventory reconciliation.
- SSE clients must recover by reconnecting and fetching current state.
- A daemon restart must preserve machine UUID, services, reservations, and history.

16. Testing and quality requirements

Test layer	Coverage
Unit	Socket normalization, process identity, provider error handling, registry, reservation validation, address overlap, conflict classification, history deduplication.
Integration	Scan and persist, process exit, partial scan, service matching, reservation conflicts, Docker lifecycle, API auth, daemon lifecycle, SSE reconnect.
GUI/TUI/CLI	Django test client; Textual pilot/screen tests; CLI command tests against API client mocks and a running test daemon.
Platform	Linux, Windows 11, macOS; TCP/UDP, IPv4/IPv6, permissions, process disappearance, platform-specific limitations.
Docker	Docker Engine on Linux and Docker Desktop on Windows/macOS where available; unavailable/permission-denied paths.

- Use controlled test sockets bound to localhost and temporary database directories.
- Tests must not depend on arbitrary ports being free without safely selecting/allocating them.
- Use mocks/fakes for process and Docker providers in deterministic unit tests.
- Run formatting/linting and tests before declaring implementation complete.
- Document which platform/Docker scenarios were actually executed versus merely supported by design.

17. Proposed package layout

```
portregistry/
  pyproject.toml
  README.md
  LICENSE
  src/portregistry/
    config.py
    paths.py
    exceptions.py
    core/
      types.py
      events.py
      identity.py
      application.py
    discovery/
      coordinator.py
      normalizer.py
      process_resolver.py
      providers/
        base.py
        psutil_provider.py
        linux.py
        windows.py
        macos.py
      docker/
        provider.py
        events.py
        normalizer.py
    services/
      registry.py
      reservations.py
      conflicts.py
      processes.py
      history.py
    daemon/
      lifecycle.py
      scheduler.py
      lock.py
      event_processor.py
      server.py
    django_app/
      settings.py
      urls.py
      asgi.py
      models/
      api/
      migrations/
    gui/
      views.py
      templates/
      static/
    tui/
      app.py
      screens/
      widgets/
    cli/
      commands.py
  tests/
    unit/
    integration/
    discovery/
    docker/
    api/
    interfaces/
```

This is a logical layout. Django app loading and Python packaging constraints take precedence; keep the implementation importable, testable, and maintainable rather than forcing a directory structure that breaks Django conventions.

18. Implementation sequence

1. Foundation: package metadata, config/data paths, logging, Django project/app wiring, SQLite settings, migrations, machine UUID, health/status.

2. Native discovery: psutil provider, normalized socket/process models, process identity, scan lifecycle, partial-error handling, deterministic tests.
3. Docker: optional SDK extra, inventory discovery, port mapping normalization, lifecycle events, reconnect/reconcile, unavailable states.
4. Daemon/API: single-instance lock, lifecycle, local auth, HTTP API, scheduler, SSE, graceful shutdown.
5. Registry/conflicts/history: service CRUD, reservations, address overlap logic, conflict explanations, meaningful event retention.
6. GUI: dashboard, ports explorer, process details, services, reservations, conflicts, history, live updates.
7. TUI and CLI: API-backed screens and commands, JSON/CSV output, SSH-friendly behaviour.
8. Release hardening: platform validation, docs, packaging, demo script, tests, error handling, v0.1 readiness.

The implementation order may be adjusted to achieve a working end-to-end vertical slice early. Do not postpone integration until the end: get daemon → scan → persist → API → CLI/GUI working as soon as practical.

19. Acceptance criteria

- A fresh install can initialize PortRegistry and start the daemon using documented commands.
- The daemon starts once, exposes health/status, and rejects or safely handles duplicate startup.
- The initial scan discovers a controlled TCP listener and UDP-bound socket on the local machine where OS permissions allow.
- Process identity uses machine UUID + PID + process creation time; PID reuse does not silently merge process instances.
- Permission failures and disappearing processes are surfaced as partial discovery, not fatal application failure.
- Docker discovery works when configured and does not break native discovery when unavailable.
- A service and reservation can be created, viewed, edited, and released through the API and at least one user interface; the same state is visible in all clients.
- Conflicts are classified and explained, including wildcard overlap and TCP/UDP distinction.
- History records meaningful state changes and avoids a new event for every unchanged polling cycle.
- GUI, TUI, and CLI communicate with the daemon; none independently scans or writes to SQLite.
- Restarting the daemon preserves user data and machine identity.
- The README documents installation, daemon lifecycle, permissions, Docker setup, limitations, and demo steps.
- Tests pass for the implemented functionality, and unsupported platform behaviours are documented honestly.

20. Risks and open decisions

Risk / decision	Implementation guidance
psutil visibility differs by OS and privilege	Design partial results and provider capability reporting; do not promise complete visibility.
IPv6 dual-stack overlap is OS-dependent	Use definite/potential conflict classification; avoid false certainty.
Docker Desktop networking differs from Linux Engine	Separate container port mappings from host socket ownership.
Daemon + Django ASGI lifecycle	Keep one process/worker; initialize scheduler exactly once in daemon lifecycle, never at import time.
SSE delivery gaps	Treat events as hints; reconnect and fetch authoritative current state.
History retention	30 days is proposed default and should be configurable.
Scan intervals	10s native / 30s Docker are proposed defaults; make configurable and avoid overlapping scans.
OS service installation	Defer; CLI-managed daemon is sufficient for first demo.

When a choice is not specified here and materially affects data integrity, security, cross-platform behaviour, or user-visible semantics, pause and ask the product owner rather than silently choosing a design. For low-impact implementation details, choose the simplest maintainable option and document it.

Appendix A — First demo checklist

- Install package in a clean virtual environment.
- Start daemon in foreground and verify health/status.
- Open GUI and TUI; verify both show the same machine and endpoint inventory.
- Start a test TCP server and bind a UDP socket; run/await scan and verify discovery.
- Register a service and reserve its expected port.
- Demonstrate occupied reservation or overlapping reservation conflict with explanation.
- If Docker is available, run a small container with a published port and verify container + mapping display.
- Stop the test process/container and verify state change/history after reconciliation.
- Restart daemon and verify persistent user data.
- Run automated tests and show documented limitations.

Appendix B — Source-of-truth rules

- This PRD takes precedence over assumptions made by an implementation agent.
- The daemon is the only owner of discovery, scheduling, application logic, and DB writes.
- Polling/reconciliation is the native discovery correctness mechanism; do not promise portable native socket events.
- Services/reservations are persistent user intent; process/socket records are observed machine state.
- Reservations are advisory and never guarantee OS-level port availability.
- Docker discovery is optional at runtime and read-only.
- No process control, Docker management, remote agents, telemetry, or cloud dependencies in MVP.

- If still uncertain after consulting this PRD and repository context, stop and ask the user.